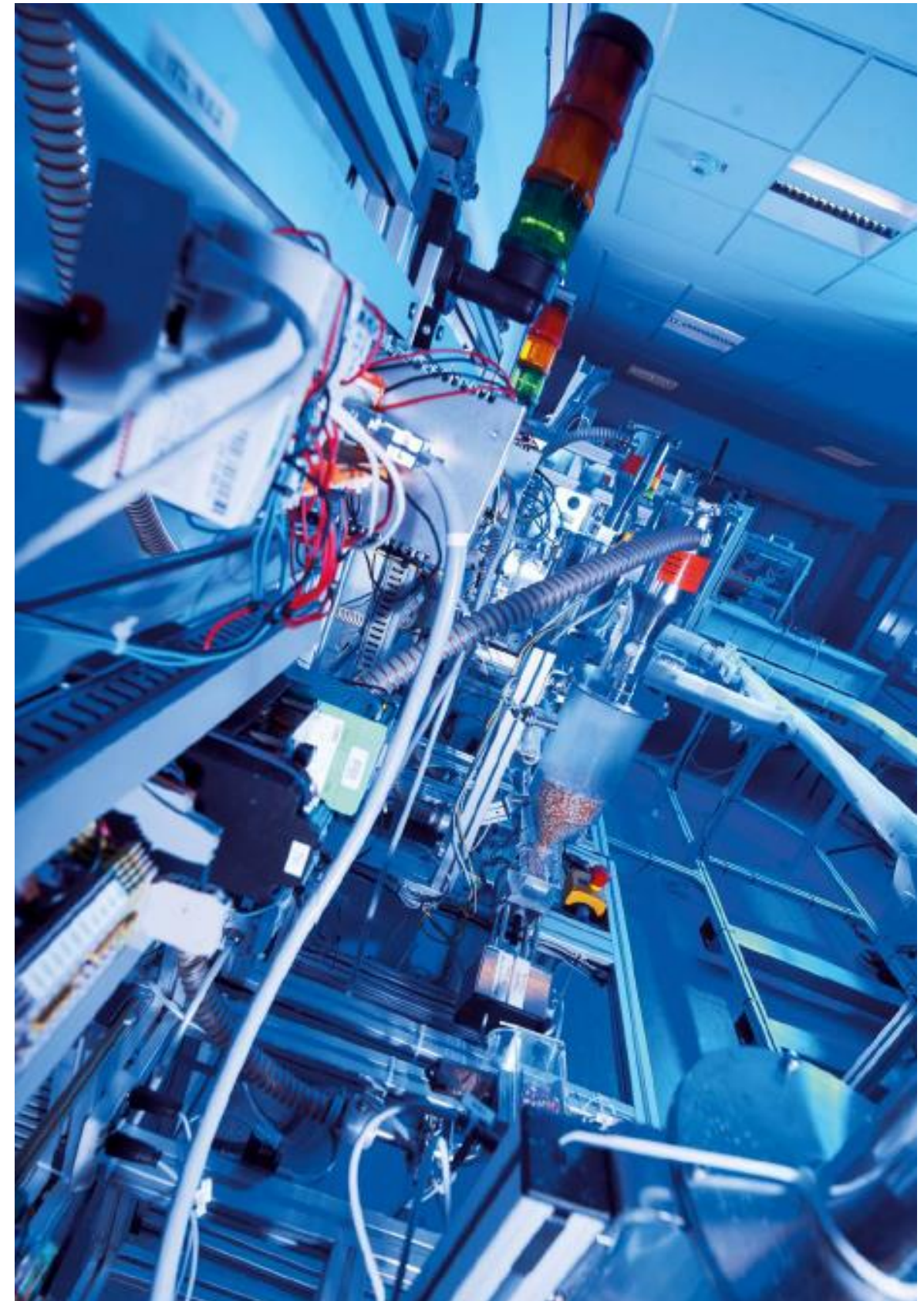


Echtzeit-Datenverarbeitung

Asynchrone
Programmierstruktur

Prof. Dr. Henning Trsek

Technische Hochschule Ostwestfalen-Lippe
Fachbereich Elektrotechnik und Technische Informatik
inIT - Institut für industrielle Informationstechnik
henning.trsek@th-owl.de

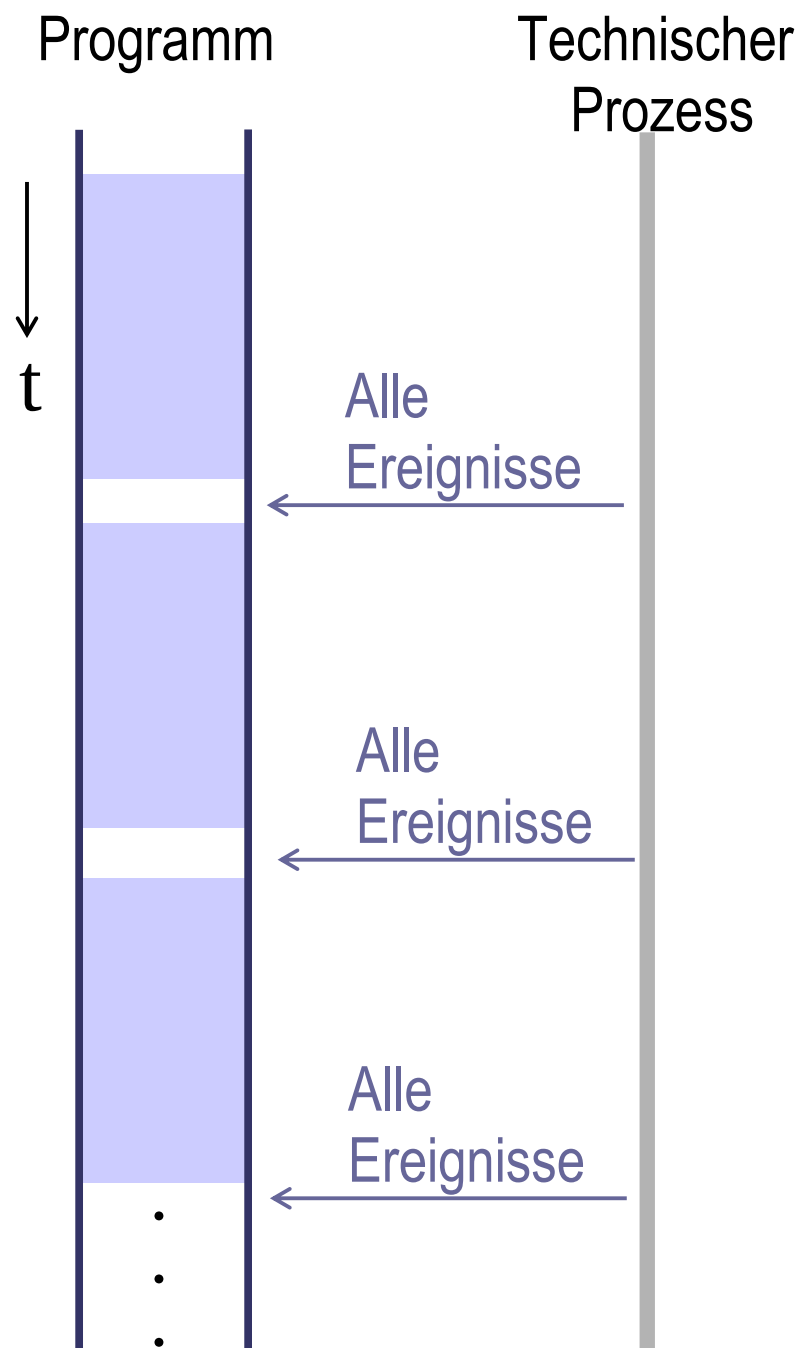


Überblick Asynchrone Programmierung

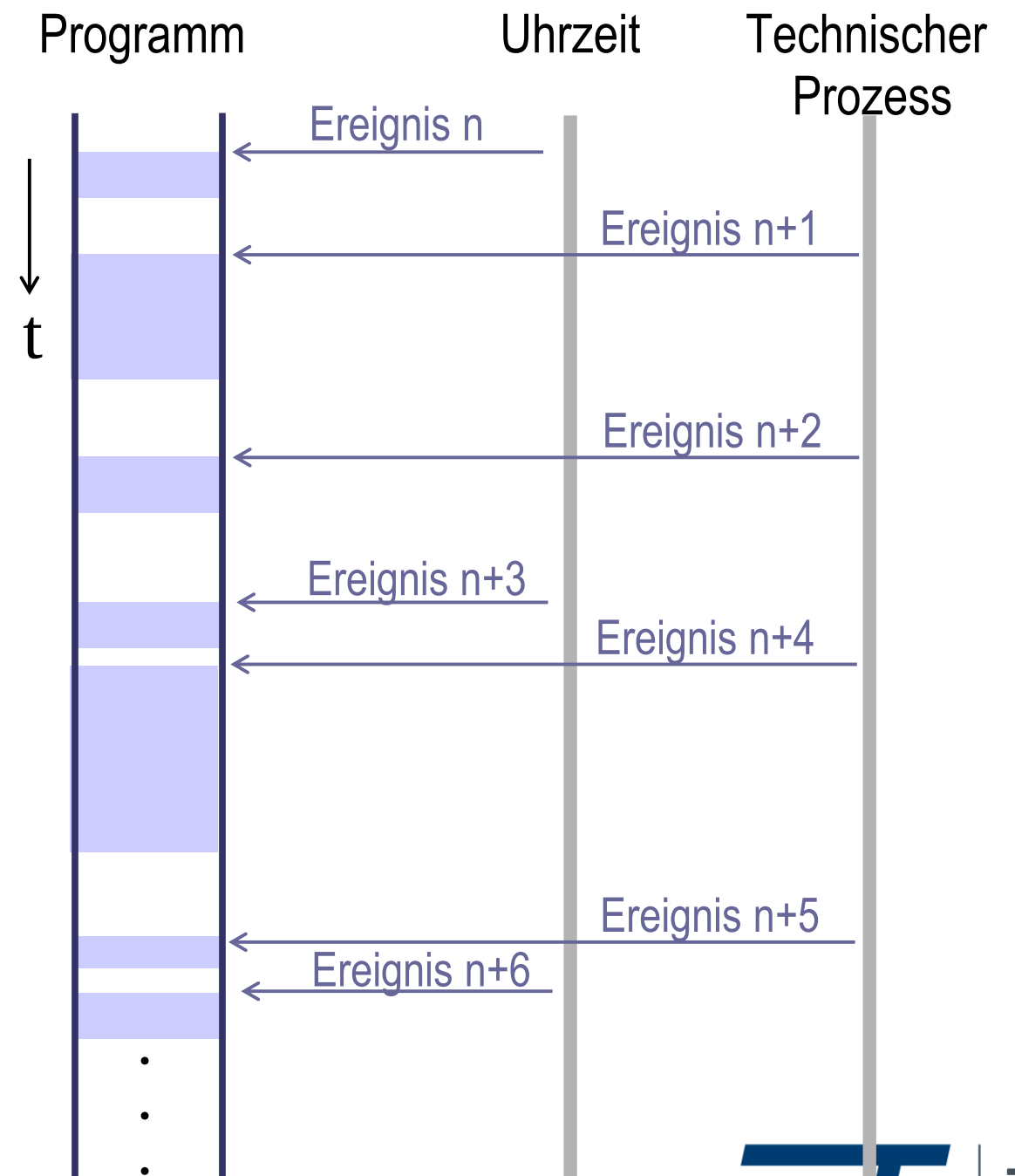
- ▶ Multitasking
 - Programmstrukturen
 - Programmstruktur in C
- ▶ Zustandsgraphen
- ▶ Taskzustandswechsel in
Multitasking-C (freeRTOS)

Asynchrone / synchrone Programmierstruktur

Synchron



Asynchron



Asynchrone Programmierstruktur

Parallele Aufgaben im Rechner (Beispiel)

messwerte_1

messwerte_2

...

regler_1

regler_2

...

protokoll

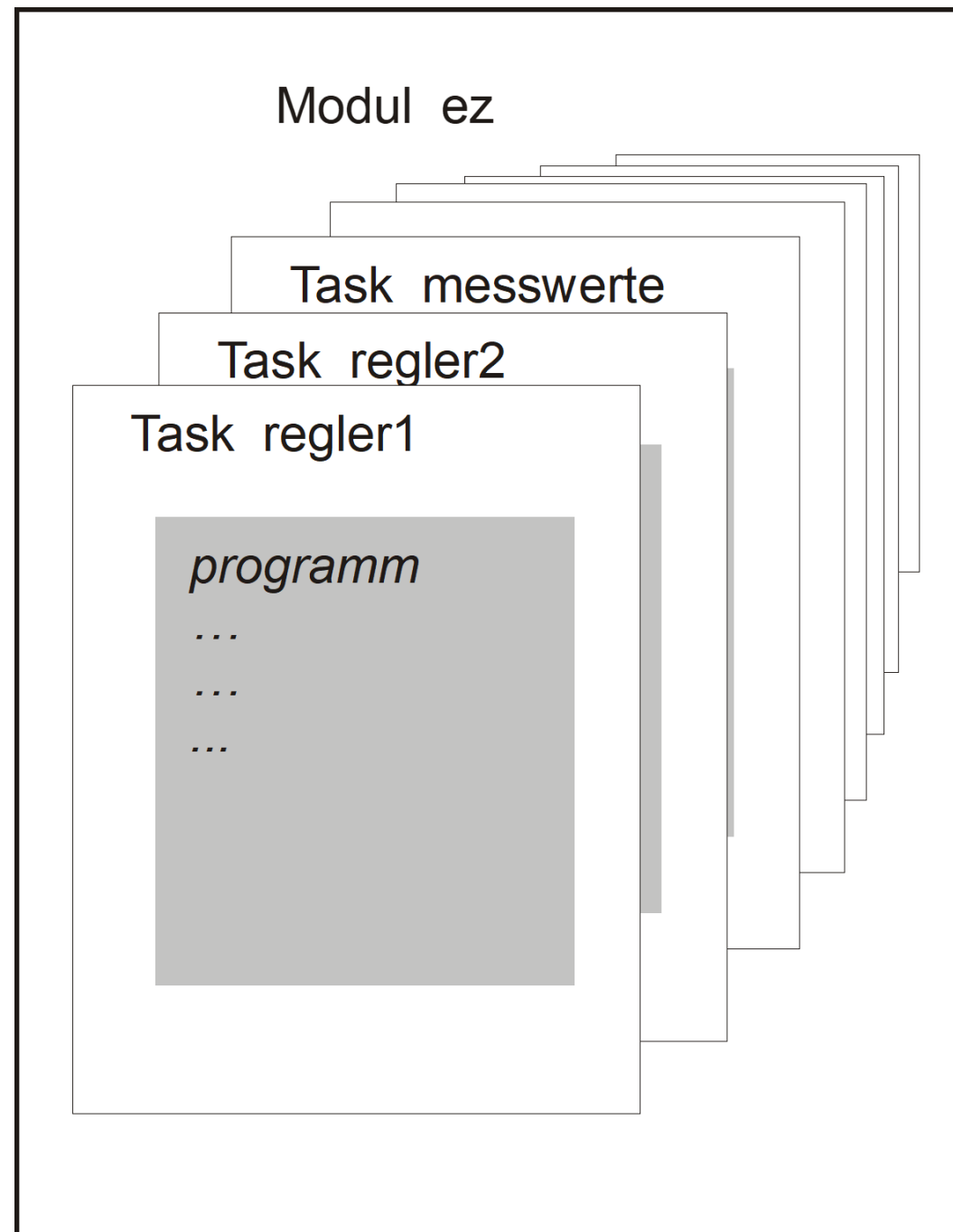
alarm

...

Asynchrone Programmierstruktur

- ▶ Parallele Aufgaben werden quasi-gleichzeitig bearbeitet
- ▶ Mehrere Hauptprogramme nebeneinander
- ▶ Multitasking Multitasking-C
- ▶ Betriebssystem: Multitasking-Echtzeit-Betriebssystem
- ▶ Real Time Operating System (RTOS)
- ▶ Taskverdrängung, preemptives Multitasking

Programmstruktur Multitasking



Modul in C ?

Mehrere unabhängige
Haupt-Programme

```

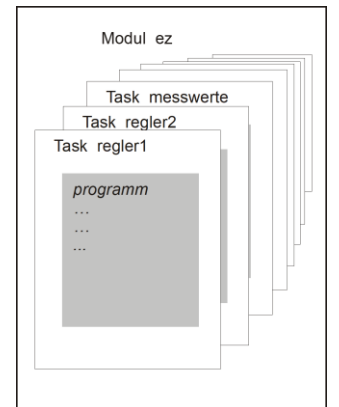
//-----
// TASK  regler1  (runtime created)
TaskHandle_t regler1_handle;
void regler1(void)
{
    ...
} // task end
//-----
// TASK  regler2  (runtime created)
TaskHandle_t regler2_handle;
void regler2(void)
{
    ...
} // task end

...

//-----
// TASK  main      (linker created)
void main(void)
{
    // create tasks
    xTaskCreate(regler1, "", 128, NULL, 30, &regler1_handle);
    xTaskCreate(regler2, "", 128, NULL, 20, &regler2_handle);
    ...
    vTaskStartScheduler();
    // main: never ending task
    for(;;);
    exit(0);
}

```

Multitasking Struktur




```

//-----
// TASK  regler1  (runtime created)
TaskHandle_t regler1_handle;
void regler1(void)
{
    ...
} // task end
//-----

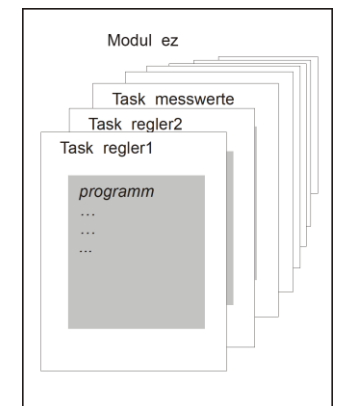
// TASK  regler2  (runtime created)
TaskHandle_t regler2_handle;
void regler2(void)
{
    ...
} // task end

...

//-----
// TASK  main      (linker created)
void main(void)
{
    // create tasks
    xTaskCreate(regler1,"", 128, NULL, 30,&regler1_handle);
    xTaskCreate(regler2,"", 128, NULL, 20,&regler2_handle);
    ...
    vTaskStartScheduler();
    // main: never ending task
    for(;;);
    exit(0);
}

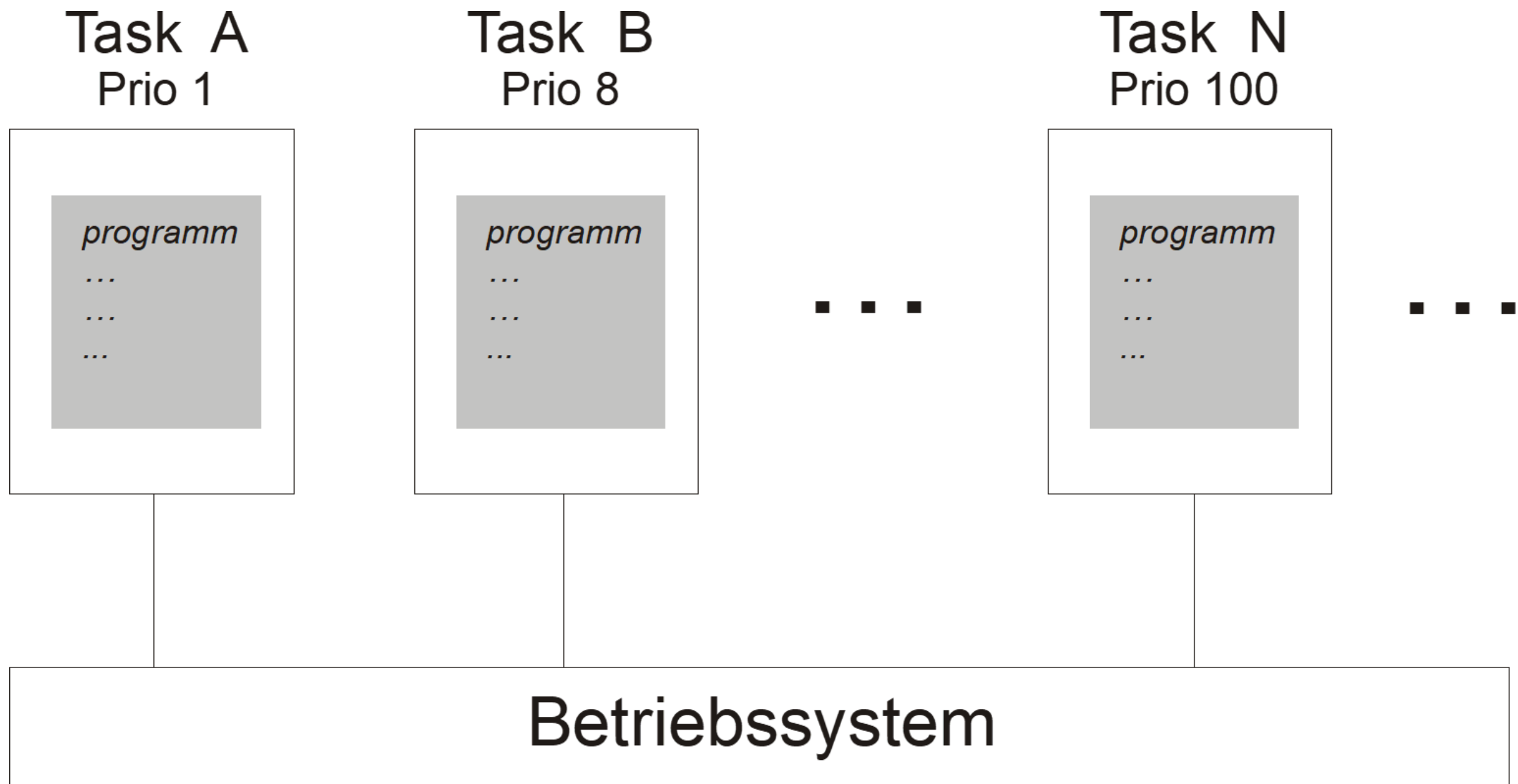
```

Multitasking Struktur

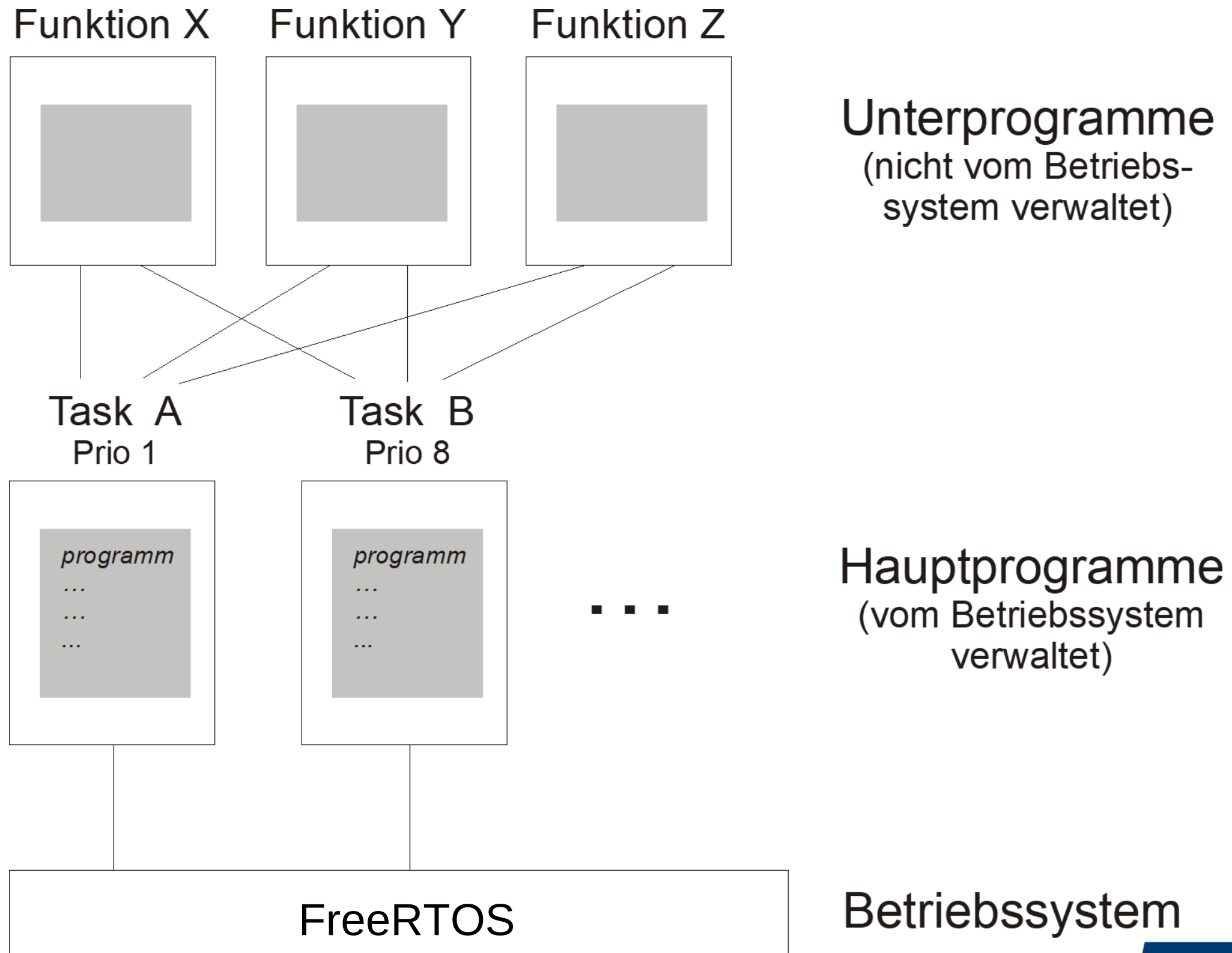


quasi-gleichzeitig

Programmstruktur Multitasking



Programmstruktur Multitasking



Multitasking

Priorität	Task
	alarm
	messwerte
	regler
	protokoll



Beispiel:

Die Protokoll-Task läuft ständig mit niedriger Priorität.

Die Task muss unterbrochen werden, wenn z.B. die Regler-Task aktiv wird.

Taskwechsel

Multitasking

Priorität Task

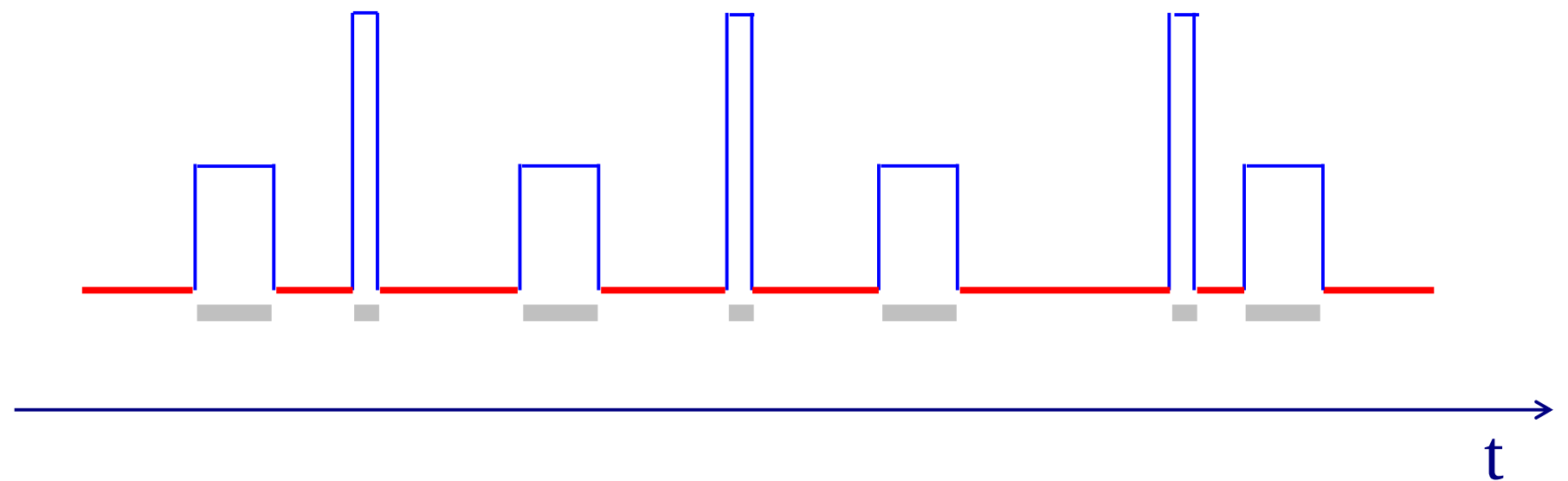


alarm

messwerte

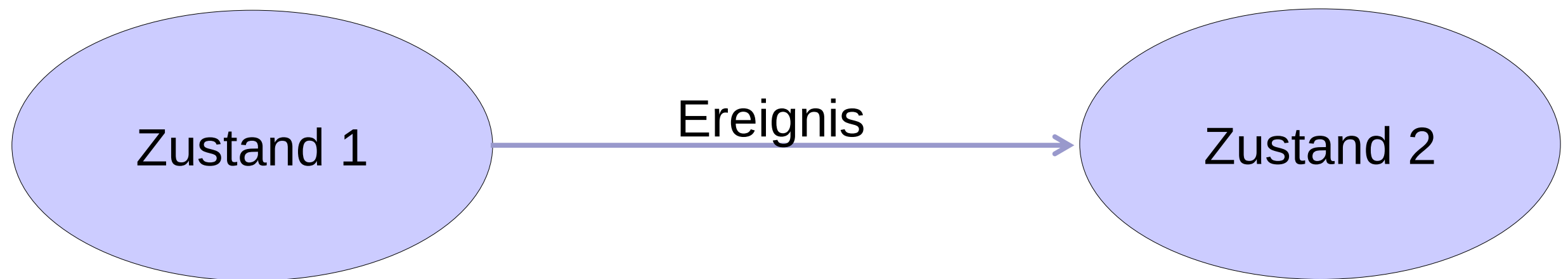
regler

protokoll



Zustandsgraphen allgemein

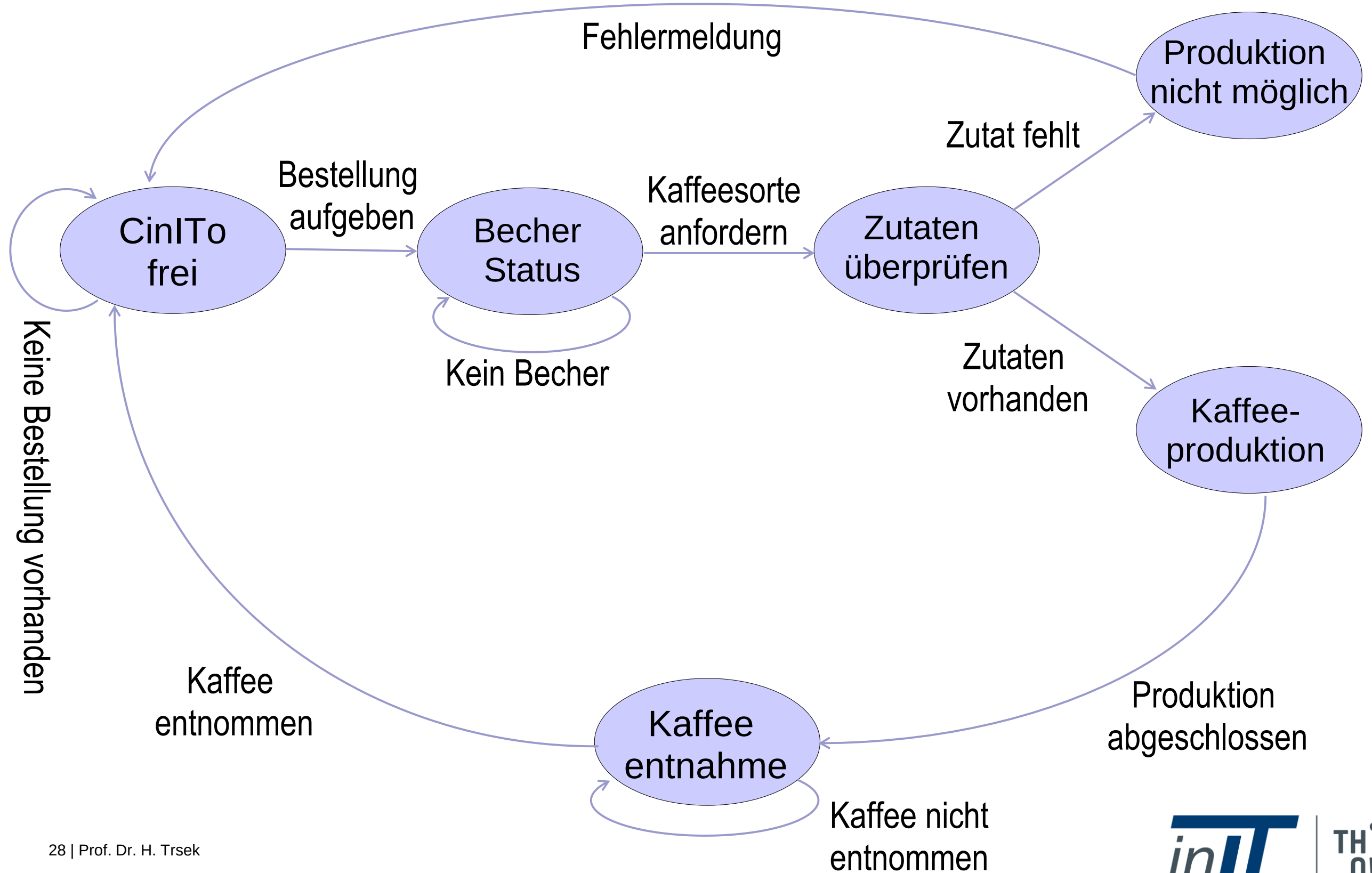
Zustandsgraph



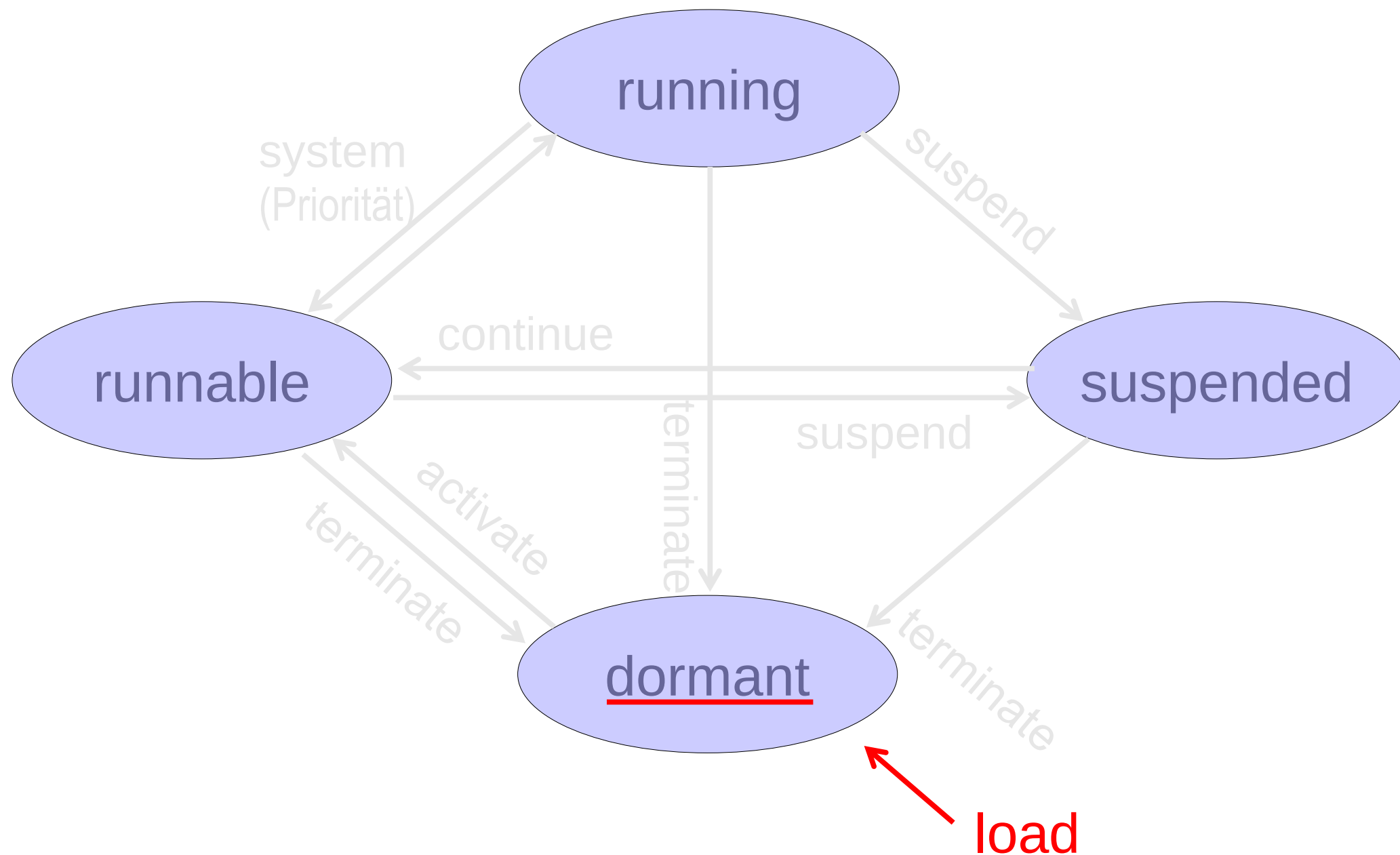
Zustandsgraph am einfachen Beispiel



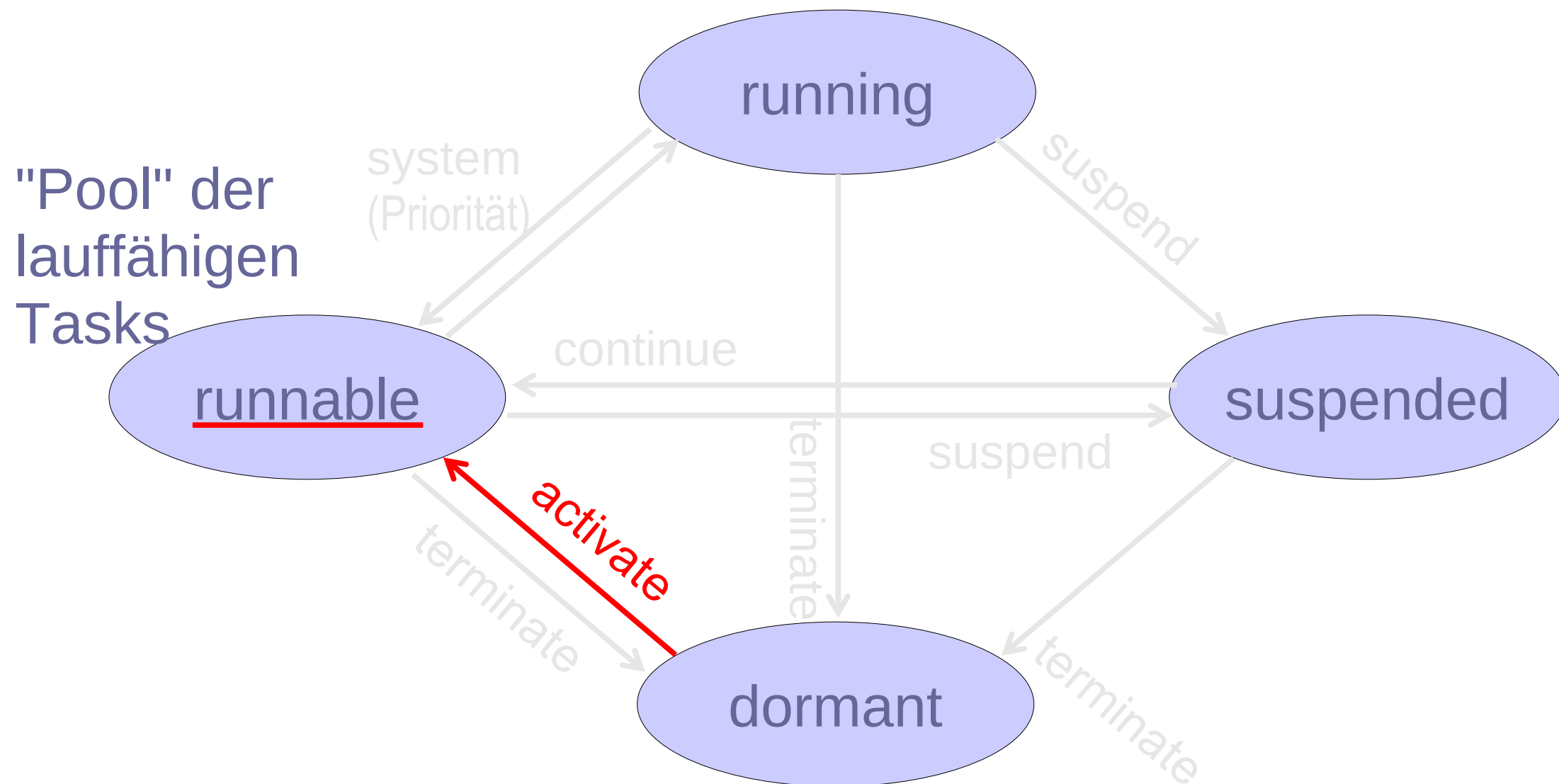
Zustandsgraph



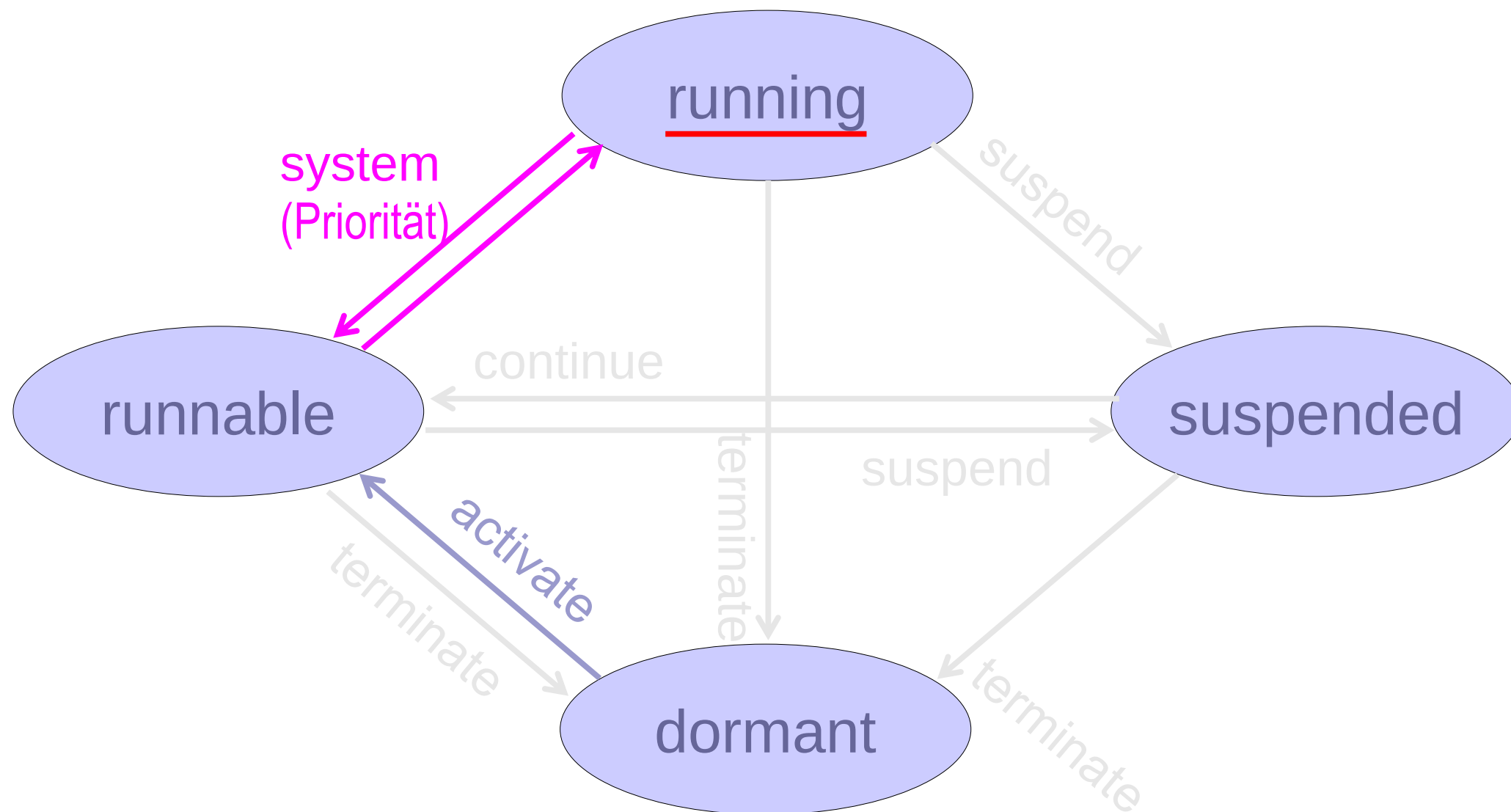
Taskzustandsgraph (vereinfacht)



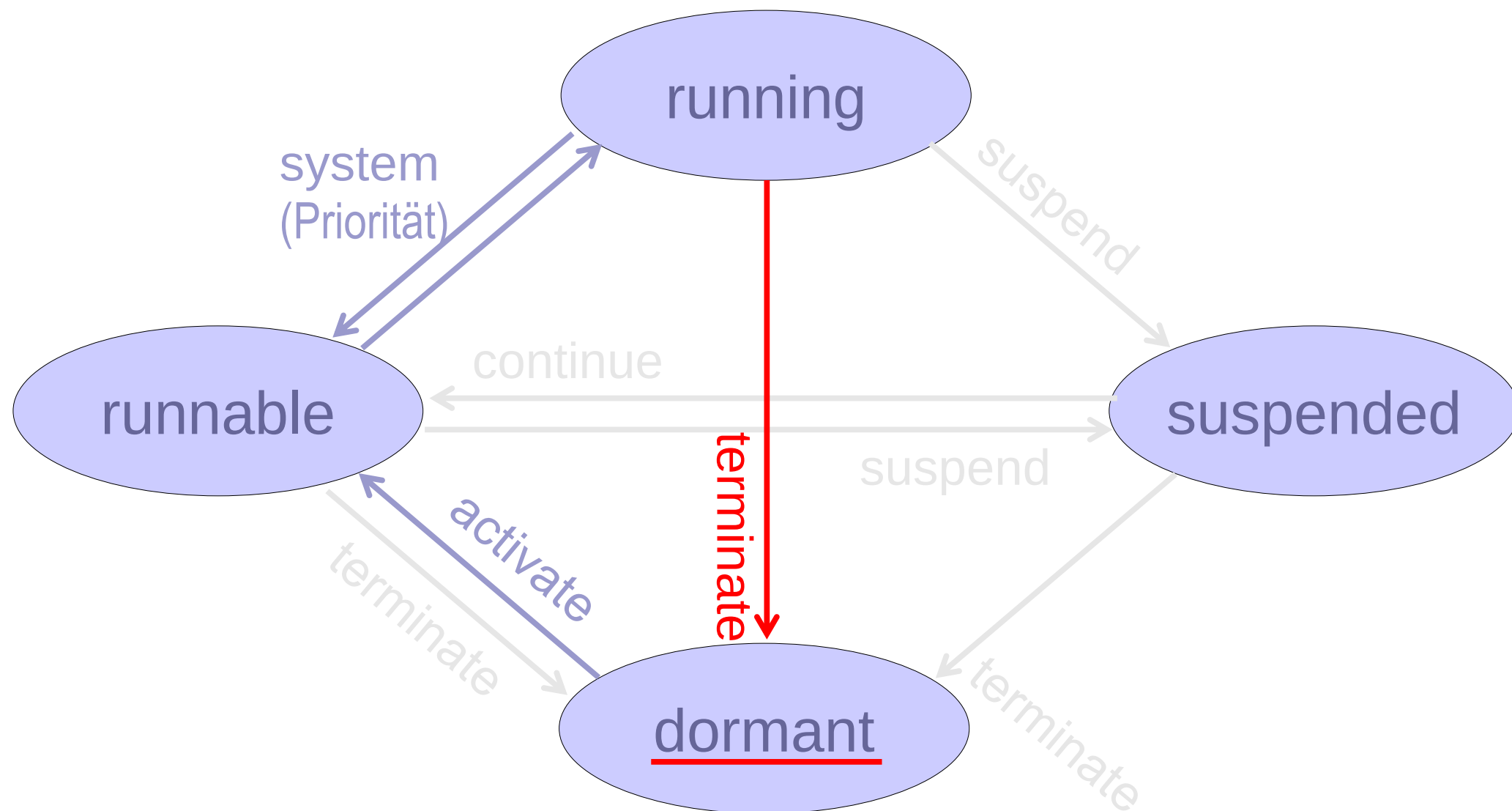
Taskzustandsgraph (vereinfacht)



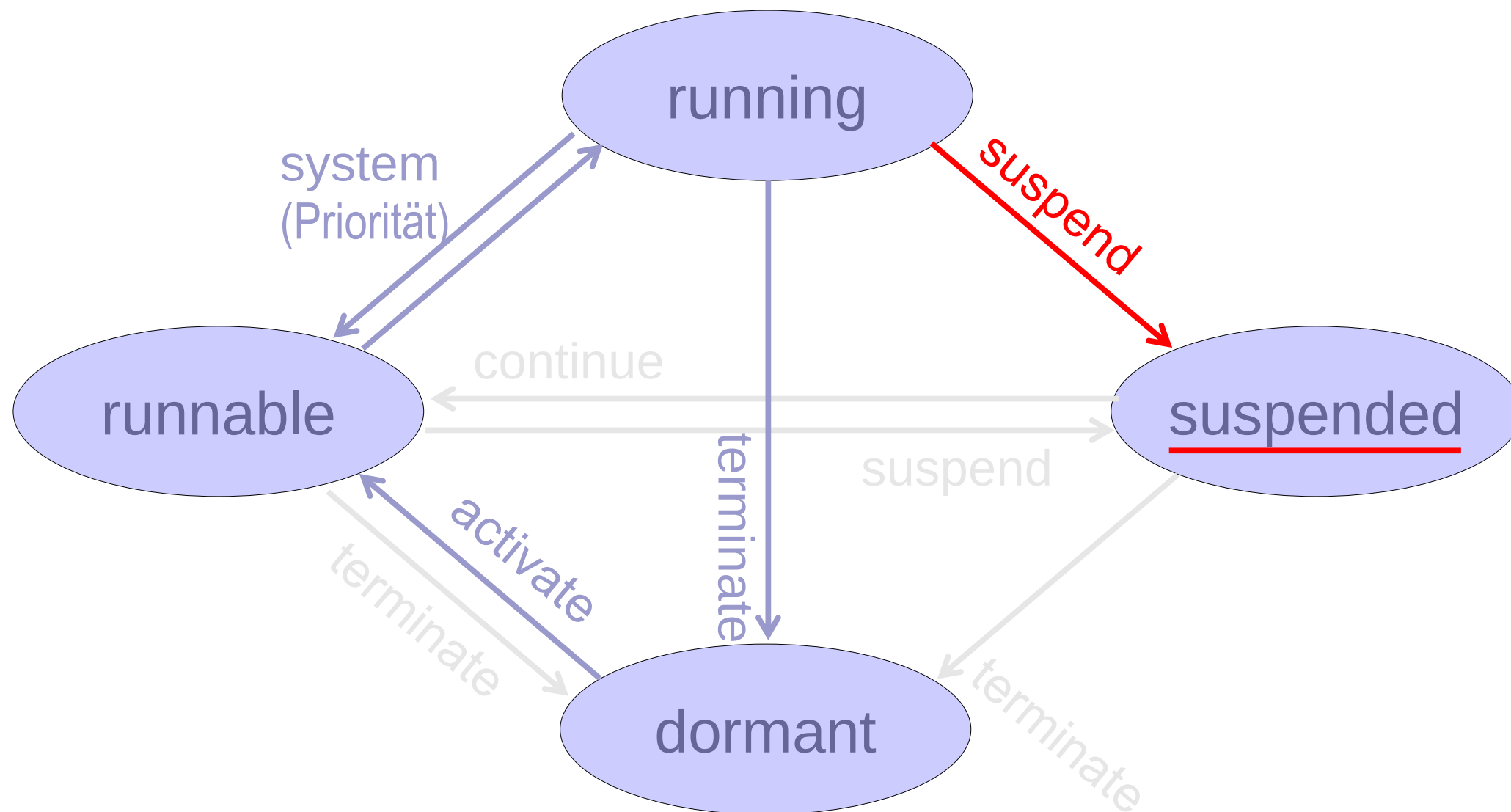
Taskzustandsgraph (vereinfacht)



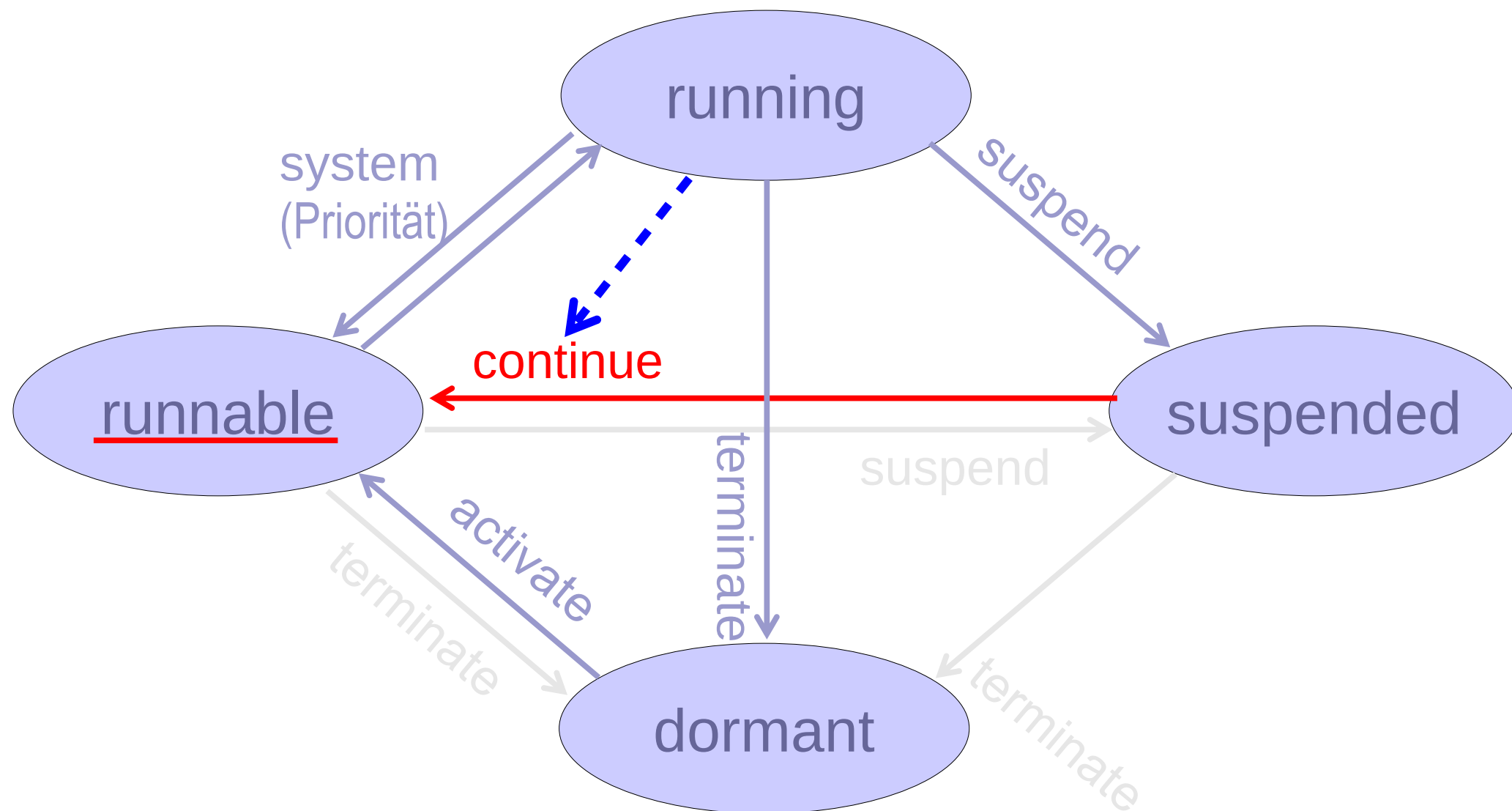
Taskzustandsgraph (vereinfacht)



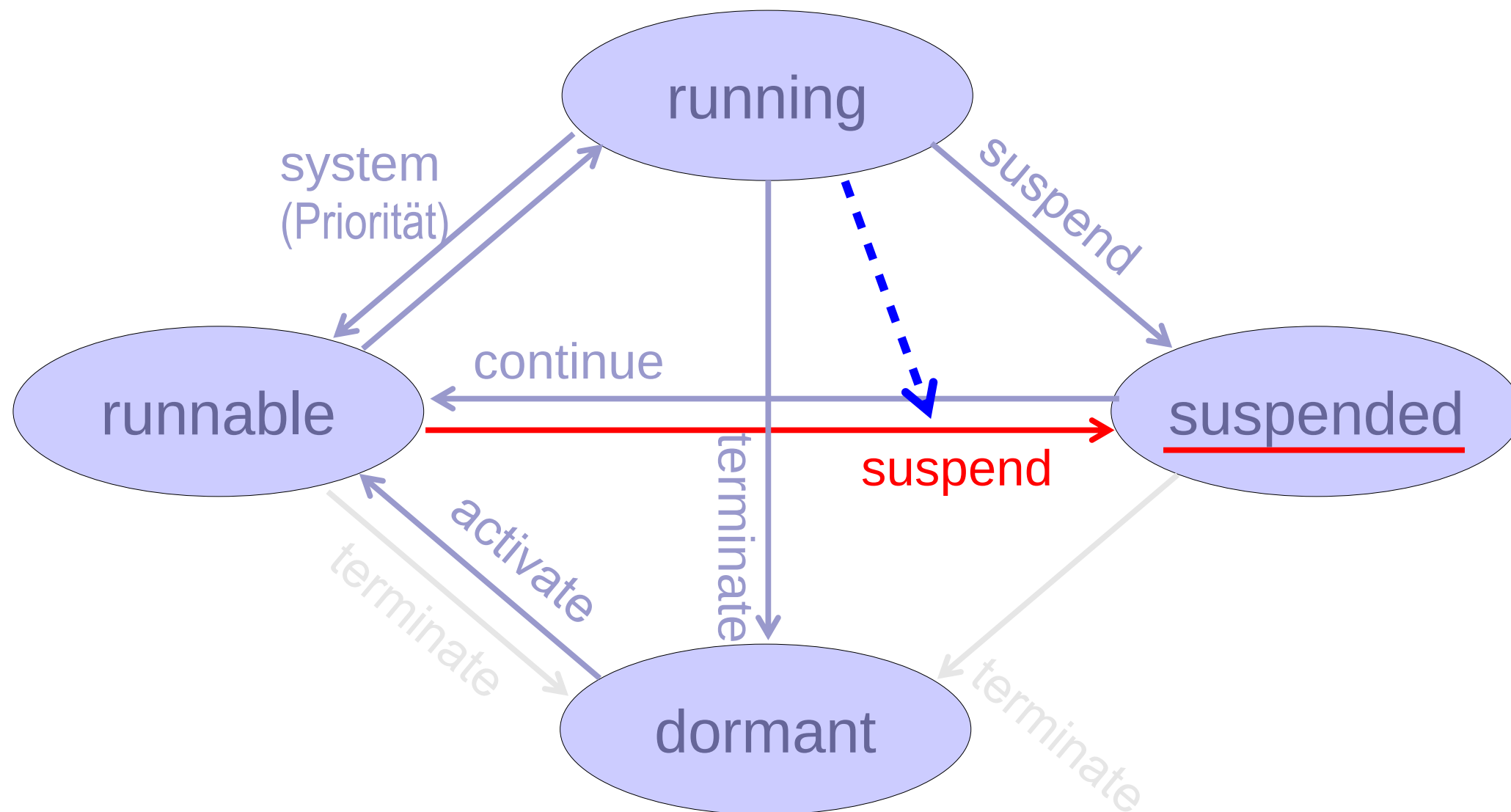
Taskzustandsgraph (vereinfacht)



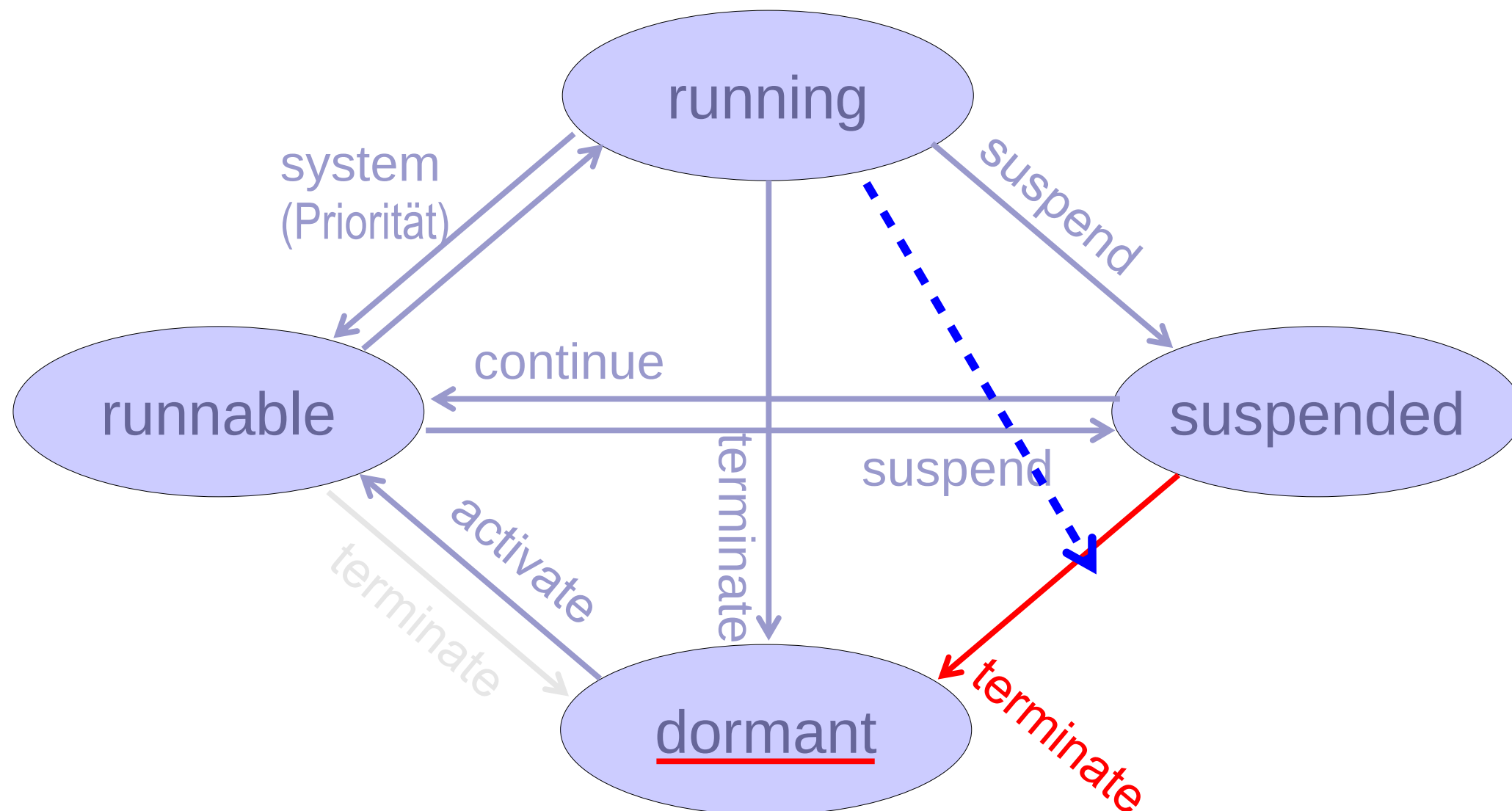
Taskzustandsgraph (vereinfacht)



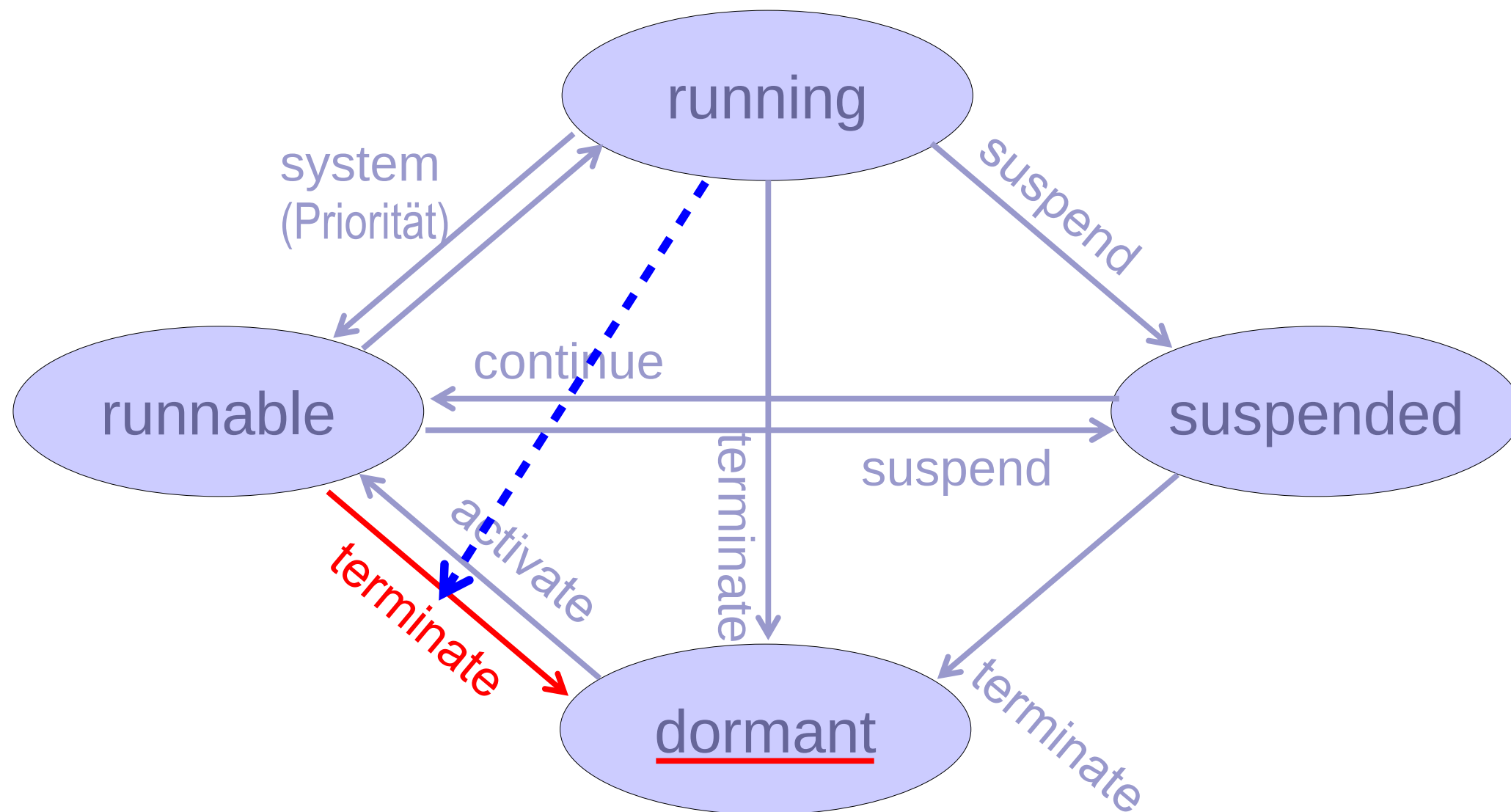
Taskzustandsgraph (vereinfacht)



Taskzustandsgraph (vereinfacht)



Taskzustandsgraph (vereinfacht)



Taskzustandswechsel in Multitasking-C (freeRTOS)

Anweisungen zum Taskzustandswechsel

```
//-----  
// TASK  ez_task_1  (runtime created)  
TaskHandle_t ez_task_1_handle  
void ez_task_1(void)  
{  
    ...  
} // task end  
//-----  
...  
  
//-----  
// TASK  main  (linker created)  
void main(void)  
{  
    // create tasks  
    xTaskCreate(ez_task_1, "", 128, NULL, 30, &ez_task_1_handle);  
    ...  
    vTaskStartScheduler();  
    ...  
  
    // main: never ending task  
    for(;;);  
    exit(0);  
}
```

Die Task wird erzeugt, befindet sich als verwaltbares Objekt im Speicher. Da der Scheduler noch nicht gestartet wurde ist die Task noch nicht aktiviert !

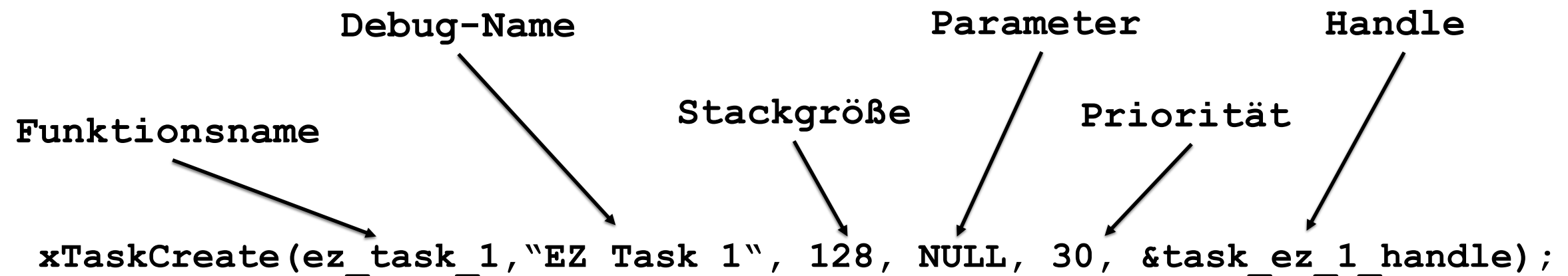
Anweisungen zum Taskzustandswechsel

```
//-----  
// TASK ez_task_1 (runtime created)  
TaskHandle_t ez_task_1_handle  
void ez_task_1(void)  
{  
    ...  
} // task end  
//-----  
...
```

Da der Scheduler bereits aktiv ist, wird die Task nach dem Erzeugen sofort ausgeführt, sofern keine höher priorisierte Task ausführbar ist!

```
//-----  
// TASK main (linker created)  
void main(void)  
{  
    // create tasks  
    ...  
    vTaskStartScheduler();  
    ...  
    xTaskCreate(ez_task_1, "", 128, NULL, 30, &ez_task_1_handle);  
  
    // main: never ending task  
    for(;;);  
    exit(0);  
}
```


Anweisungen zum Taskzustandswechsel



1. Name der von der Task auszuführenden Funktion
2. Alternativer Name zu Anzeige im Debugger
3. Größe des der Task zugewiesenen Stacks
Angabe in Wörtern (für stm32 1 Wort = 4 Byte = 32 Bit)
4. Parameter, die der Funktion aus 1. übergeben werden
5. Priorität mit der die Task vom Scheduler behandelt werden soll
6. Adresse des der Task zugewiesenen Handles
Dieser ermöglicht eine Steuerung der Task von außen.

Anweisungen zum Taskzustandswechsel

```
//-----  
// TASK ez_task_1 (runtime created)  
TaskHandle_t ez_task_1_handle  
void ez_task_1(void)  
{  
    ...  
} // task end  
//-----  
...
```

Die Task `ez_task_1` läuft nach starten des Schedulers mit Priorität 30.

```
//-----  
// TASK main (linker created)  
void main(void)  
{  
    // create tasks  
    xTaskCreate(ez_task_1, "", 128, NULL, 30, &regler1_handle);  
    ...  
    vTaskStartScheduler();  
    ...  
    // main: never ending task  
    for(;;);  
    exit(0);  
}
```

Die Task `ez_task_1` wird vom Scheduler bearbeitet, falls keine höherpriorisierte Task lauffähig ist.

Anweisungen zum Taskzustandswechsel

```
//-----
```

```
// TASK ez_task_1 (runtime created)
```

```
TaskHandle_t ez_task_1_handle
```

```
void ez_task_1(void)
```

```
{
```

```
    ...
```

```
} // task end
```

```
//-----
```

```
...
```

```
//-----
```

```
// TASK main (linker created)
```

```
void main(void)
```

```
{
```

```
    // create tasks
```

```
    xTaskCreate(ez_task_1, "", 128, NULL, 30, &regler1_handle);
```

```
    ...
```

```
    vTaskStartScheduler();
```

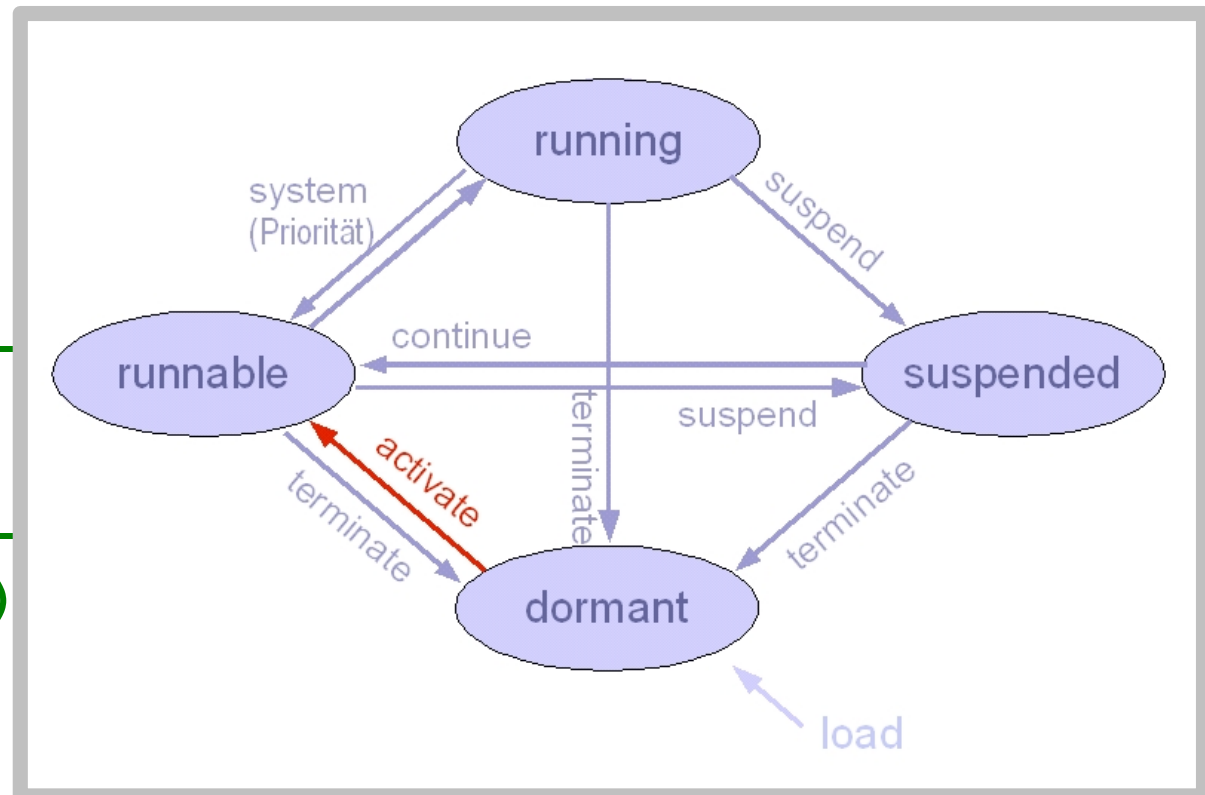
```
    ...
```

```
    // main: never ending task
```

```
    for(;;);
```

```
    exit(0);
```

```
}
```



Die Task `ez_task_1` wird vom Prozessor bearbeitet, falls keine höherpriorisierte Task lauffähig ist.

Anweisungen zum Taskzustandswechsel

```
//-----  
// TASK  ez_task_1    (runtime created) [Priorität 30]  
TaskHandle_t ez_task_1_handle  
void ez_task_1(void)  
{  
    ...  
    vTaskDelete(ez_task_2_handle);  
    vTaskDelete();  
    ...  
} // task end  
  
//-----  
// TASK  ez_task_2    (runtime created) [Priorität 20]  
TaskHandle_t ez_task_2_handle  
void ez_task_2(void)  
{  
    for (;;) { ... }  
} // task end  
//-----
```

Prototyp:

```
void vTaskDelete( TaskHandle_t handle );
```

handle = NULL → Der Task beendet sich selbst

Anweisungen zum Taskzustandswechsel

```
//-----  
// TASK  ez_task_1    (runtime created) [Priorität 30]  
TaskHandle_t ez_task_1_handle  
void ez_task_1(void)  
{  
    ...  
    vTaskDelete(ez_task_2_handle);  
    vTaskDelete();  
    ...  
} // task end  
  
//-----  
// TASK  ez_task_2    (runtime created) [Priorität 20]  
TaskHandle_t ez_task_2_handle  
void ez_task_2(void)  
{  
    for (;;) {  
    } // task end  
} //-----
```

Prototyp:

```
void vTaskDelete( TaskHandle_t handle );
```

handle = NULL → Der Task beendet sich selbst

Anweisungen zum Taskzustandswechsel

```
//-----  
// TASK ez_task_1 (runtime created) [Priorität 30]  
TaskHandle_t ez_task_1_handle  
void ez_task_1(void)  
{  
    ...  
    vTaskDelete(ez_task_2_handle);  
    vTaskDelete();  
    ...  
} // task end  
  
//-----  
// TASK ez_task_2 (runtime created) [Priorität 20]  
TaskHandle_t ez_task_2_handle  
void ez_task_2(void)  
{  
    for (;;) { ... }  
} // task end  
//-----
```

Task terminieren:
Die Task `ez_task_2` wird sofort beendet.

Prototyp:

```
void vTaskDelete( TaskHandle_t handle );  
handle = NULL → Der Task beendet sich selbst
```

Anweisungen zum Taskzustandswechsel

```
//-----  
// TASK  ez_task_1    (runtime created) [Priorität 30]  
TaskHandle_t ez_task_1_handle  
void ez_task_1(void)  
{  
    ...  
    vTaskSuspend();  
    ...  
} // task end  
  
//-----
```



Task suspendieren:
Die Task **ez_task_1** blockiert sich selbst
auf unbestimmte Zeit.

Prototyp:

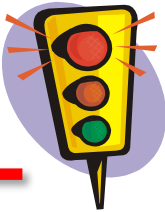
```
void vTaskSuspend( TaskHandle_t );
```

```
void vTaskResume ( TaskHandle_t );
```

handle = NULL → Ziel ist die ausführende Task

Anweisungen zum Taskzustandswechsel

```
//-----  
// TASK ez_task_1 (runtime created) [Priorität 30]  
TaskHandle_t ez_task_1_handle  
void ez_task_1(void)  
{  
    ...  
    vTaskSuspend();  
    ...  
} // task end  
  
//-----  
// TASK ez_task_2 (runtime created) [Priorität 20]  
TaskHandle_t ez_task_2_handle  
void ez_task_2(void)  
{  
    ...  
    vTaskResume(ez_task_1_handle);  
    ...  
} // task end  
//-----
```



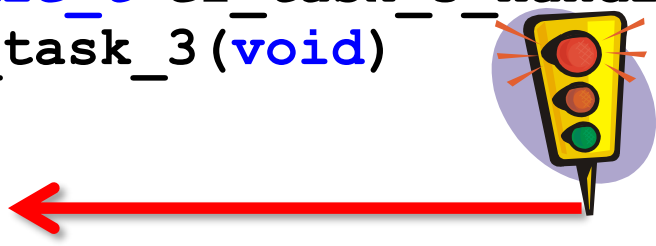
Task weiterlaufen lassen:
Die Blockierung für Task `ez_task_1`
wird aufgehoben.

Prototyp:

```
void vTaskSuspend( TaskHandle_t );  
void vTaskResume ( TaskHandle_t );  
handle = NULL → Ziel ist die ausführende Task
```


Anweisungen zum Taskzustandswechsel

```
//-----  
// TASK  ez_task_3  (runtime created) [Priorität 10]  
TaskHandle_t ez_task_3_handle  
void ez_task_3(void)  
{  
    ...  
    ...  
} // task end  
//-----  
// TASK  ez_task_2  (runtime created) [Priorität 20]  
TaskHandle_t ez_task_2_handle  
void ez_task_2(void)  
{  
    ...  
    vTaskSuspend(ez_task_3_handle);  
    ...  
    vTaskResume(ez_task_3_handle);  
    ...  
} // task end  
//-----
```



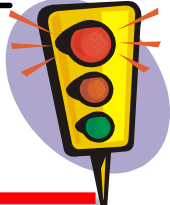
Task suspendieren:
Die Task `ez_task_3` wird blockiert
auf unbestimmte Zeit.

Prototyp:

```
void vTaskSuspend( TaskHandle_t );  
void vTaskResume ( TaskHandle_t );  
handle = NULL → Ziel ist die ausführende Task
```

Anweisungen zum Taskzustandswechsel

```
//-----  
// TASK ez_task_3 (runtime created) [Priorität 10]  
TaskHandle_t ez_task_3_handle  
void ez_task_3(void)  
{  
    ...  
    ...  
} // task end  
//-----  
// TASK ez_task_2 (runtime created) [Priorität 20]  
TaskHandle_t ez_task_2_handle  
void ez_task_2(void)  
{  
    ...  
    vTaskSuspend(ez_task_3_handle);  
    ...  
    vTaskResume(ez_task_3_handle);  
} // task end  
//-----
```



Task weiterlaufen lassen:
Die Blockierung für Task `ez_task_3`
wird aufgehoben.

Prototyp:

```
void vTaskSuspend( TaskHandle_t );  
void vTaskResume ( TaskHandle_t );  
handle = NULL → Ziel ist die ausführende Task
```